

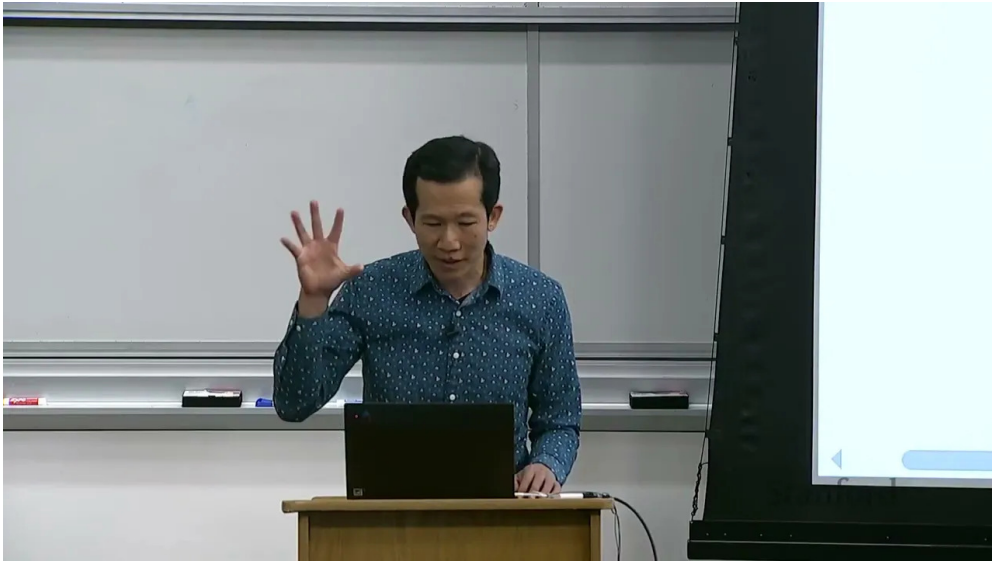
课程笔记

CS336 Lecture 2: PyTorch 与资源核算

Stanford CS336: Language Modeling from Scratch (Spring 2025)

讲师: Percy Liang 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 79 分钟

视频链接: <https://www.youtube.com/watch?v=msHyYioAyNE>

Contents

1	开场：为什么需要资源核算	2
1.1	本课定位	2
1.2	用 napkin math 回答两个问题	2
1.3	为什么要做 resource accounting	3
2	内存核算：Tensor 与浮点数	3
2.1	Tensor：深度学习的一切都存在于这里	3
2.2	浮点数格式	4
2.2.1	FP32 (float32 / single precision)	5
2.2.2	FP16 vs BF16	5
2.3	Tensor 的内存占用	6
2.3.1	混合精度训练	7
2.4	Views、Strides 与 Contiguous	7
2.5	Broadcasting	9
3	计算图与 Autograd	9
3.1	Autograd：自动微分	9
3.2	计算图：leaf variables	10
4	构建模型：nn.Module	11
4.1	nn.Linear：最小的模块	11
4.2	nn.Module：组合的基石	12
4.3	Einsum：给维度起名字	12
4.4	训练循环	13
5	算力核算：从 matmul 到 6ND	14
5.1	矩阵乘法的 FLOPs	14
5.2	从 matmul 到 FLOPs/s 再到 MFU	15
5.3	Linear Layer 的 FLOPs：前向与反向	17
5.3.1	前向 FLOPs	17
5.3.2	反向 FLOPs：链式法则	17
5.3.3	推广到 Transformer	18
5.4	Attention 的 FLOPs	19
6	内存核算：四类占用	19
6.1	参数与梯度	19
6.2	激活内存	20
6.3	优化器状态内存	21
6.4	总内存公式	21
7	Optimizer 的实现	22
8	Gradient Checkpointing 与实用技巧	23
8.1	Gradient Checkpointing：用计算换内存	23
8.2	Checkpointing（模型保存）	23
8.3	混合精度的实践建议	24
9	总结	24
9.1	核心要点	24
9.2	与 Assignment 1 的衔接	24
9.3	下节预告	24
9.4	配套阅读	25

1 开场：为什么需要资源核算

1.1 本课定位

承接第一讲的课程导论，Percy Liang 在本讲开篇明确指出本节课的两个目标。第一，过一遍构建模型所需的 PyTorch 原语（primitives）：从 tensors（张量）出发，构建 models（模型）、optimizers（优化器）和 training loop（训练循环）。第二，并贯穿始终的核心是效率（efficiency）——具体来说，我们究竟如何使用资源，包括 memory（内存）与 compute（算力）。

本讲的知识类型

对应第一讲的“三类知识”框架：

- 机制（Mechanics）：本讲主体，理解 PyTorch 在原语层如何运作，是可传授、可迁移的。
- 心态（Mindset）：resource accounting（资源核算）本身就是一种心态——它不难，但你必须去做。
- 直觉（Intuition）：本讲基本不涉及，重点在机制与心态的夯实。

Percy 特别强调，本讲不会详细讲 Transformer 架构（那是下一讲 Tatsu 的内容），而是用更简单的线性模型来讲清楚原语和资源核算。理解 Transformer 的途径有很多：Assignment 1 的 handout、网上的图文教程。本讲的焦点是：每一份 FLOPs 去了哪里、每一字节内存被谁占用。

1.2 用 napkin math 回答两个问题

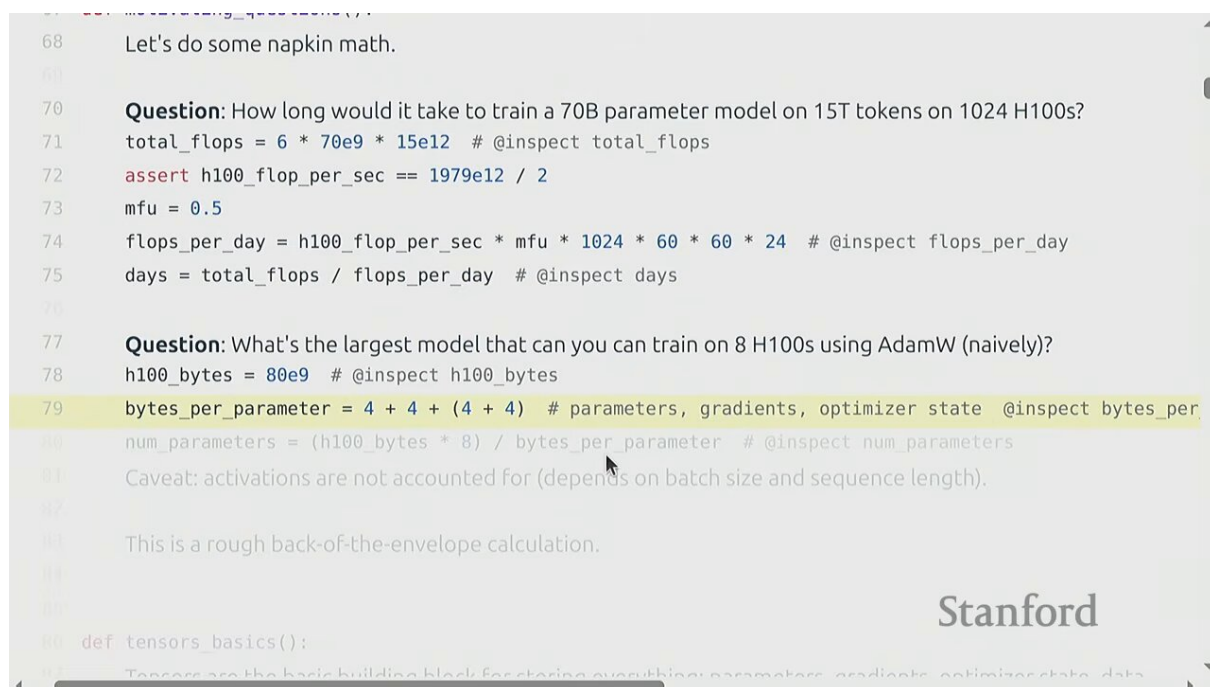


Figure 1: 开场 motivating questions：用 napkin math 估算训练成本¹

Percy 抛出两个用“餐巾纸背面算术”（napkin math）就能回答的问题，它们正是 resource accounting 的典型应用：

两个 motivating questions

- 问题一：用 1024 块 H100 训练一个 700 亿参数的 dense transformer，喂 15 万亿 token，需要多久？
问题二：在 8 块 H100 上用 AdamW 训练（不做太花哨的优化），能训的最大模型是多大？

¹视频画面时间区间：02:45 – 03:15。

问题一的解答思路如下。先算训练所需的总 FLOPs:

$$\text{FLOPs}_{\text{train}} = 6 \times N_{\text{params}} \times N_{\text{tokens}}$$

- $N_{\text{params}} = 70 \times 10^9$: 模型参数量
- $N_{\text{tokens}} = 15 \times 10^{12}$: 训练 token 数
- 系数 6: 每个参数在每个 token 上大约做 6 次 FLOPs (前向 2 + 反向 4, 本讲后会推导)

再算硬件每天能提供的 FLOPs。H100 的峰值算力约为 $P = 989 \times 10^{12}$ FLOPs/s (BF16), 设 MFU (model FLOPs utilization) 为 0.5, 则 1024 块卡一天能算:

$$\text{FLOPs}_{\text{per day}} = 1024 \times 989 \times 10^{12} \times 0.5 \times 86400$$

两者相除即得训练天数:

$$T_{\text{days}} = \frac{\text{FLOPs}_{\text{train}}}{\text{FLOPs}_{\text{per day}}} \approx 144 \text{ 天}$$

问题二则从内存角度切入。H100 有 80GB HBM, 用 AdamW 时每参数约需 16 字节 (参数 4 + 梯度 4 + 优化器状态 m, v 各 4, 后文详述), 于是:

$$N_{\text{params}}^{\text{max}} = \frac{8 \times 80 \times 10^9}{16} \approx 40 \times 10^9$$

这是粗略估算

这个估算没有算 activations (激活), 而激活占用取决于 batch size 和 sequence length。本讲用线性模型回避了这点, 但 Assignment 1 的 Transformer 必须把它算进去。

1.3 为什么要做 resource accounting

效率是核心目标

Percy 直言: 你可能习惯了“实现一个模型、训练它、发生什么是什么”。但记住效率是这场游戏的名字 (efficiency is the name of the game)。要高效, 你必须精确知道自己到底消耗了多少 FLOPs——因为当这些数字变大时, 它们直接换算成美元, 你希望它尽可能小。

本讲的思路是: 先讲 memory accounting (内存核算), 再讲 compute accounting (算力核算), 最后自底向上把模型搭起来。

2 内存核算: Tensor 与浮点数

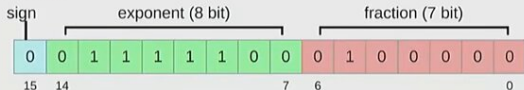
2.1 Tensor: 深度学习的一切都存在于这里

Tensor (张量) 是深度学习中存储一切的构建块 (building block): parameters (参数)、gradients (梯度)、optimizer state (优化器状态)、data (数据)、activations (激活), 都是 tensor。你可以用多种方式创建 tensor, 也可以创建一个未初始化的 tensor 然后用特殊初始化方法填充参数。

133 `assert x == 0 # Underflow!`
 134 If this happens when you train, you can get instability.

136 **bfloat16**
 137 [\[Wikipedia\]](#)

138 **bfloat16**



139 Google Brian developed bfloat (brain floating point) in 2018 to address this issue.
 140 bfloat16 uses the same memory as float16 but has the same dynamic range as float32!
 141 The only catch is that the resolution is worse, but this matters less for deep learning.
 142 `x = torch.tensor([1e-8], dtype=torch.bfloat16) # @inspect x`
 143 `assert x != 0 # No underflow!`

144 Let's compare the dynamic ranges and memory usage of the different data types:
 145 `float32_info = torch.finfo(torch.float32) # @inspect float32_info`
 146 `float16_info = torch.finfo(torch.float16) # @inspect float16_info`
 147 `bfloat16_info = torch.finfo(torch.bfloat16) # @inspect bfloat16_info`

Stanford

Figure 2: Tensor 内存计算：一个 4×8 的 float32 矩阵占 128 字节²

2.2 浮点数格式

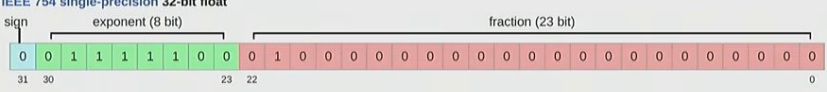
我们感兴趣的每个 tensor 几乎都以 floating-point number（浮点数）形式存储。浮点数有多种表示方式，理解它们的位结构是内存核算的基础。

99 `nn.init.trunc_normal_(x, mean=0, std=1, a=-2, b=2) # @inspect x`

102 `def tensors_memory():`
 103 Almost everything (parameters, gradients, activations, optimizer states) are stored as floating point numbers.

105 **float32**
 106 [\[Wikipedia\]](#)

107 **IEEE 754 single-precision 32-bit float**



108 The float32 data type (also known as fp32 or single precision) is the default.
 109 Traditionally, in scientific computing, float32 is the baseline; you could use double precision (float64) in some cases.
 110 In deep learning, you can be a lot sloppier.

111 Let's examine memory usage of these tensors.
 112 Memory is determined by the (i) number of values and (ii) data type of each value.
 113 `x = torch.zeros(4, 8) # @inspect x`

Stanford

Figure 3: FP32（单精度）位结构：1 sign + 8 exponent + 23 fraction = 32 bits³

²视频画面时间区间：10:00 – 10:30。

³视频画面时间区间：06:45 – 07:15。

2.2.1 FP32 (float32 / single precision)

最默认的方式是 float32 (FP32)，共 32 位：1 位 sign (符号)、8 位 exponent (指数)、23 位 fraction (尾数/小数部分)。Exponent 决定 dynamic range (动态范围)，fraction 决定能表示多少不同的值。FP32 也叫 single precision (单精度)，是计算的“黄金标准”，机器学习里常称为 full precision (全精度)。

“full precision”是相对的

科学计算的人听到 FP32 叫“全精度”会笑——他们用 float64 甚至更高。但对机器学习而言，FP32 几乎是你需要的上限，因为深度学习本身就是“sloppy”（粗放）的。

2.2.2 FP16 vs BF16

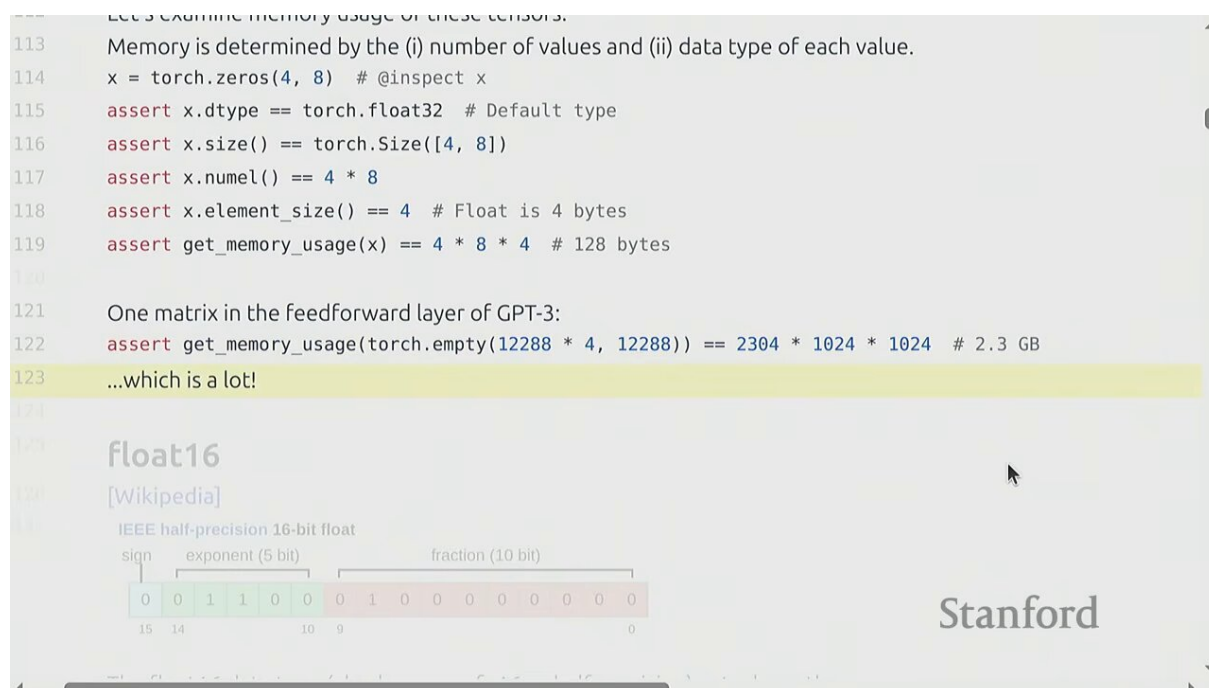


Figure 4: FP16 vs BF16 对比：FP16(1+5+10) 动态范围小易溢出，BF16(1+8+7) 与 FP32 同动态范围⁴

除了 FP32，还有两种 16 位格式。FP16 把 exponent 从 8 缩到 5、fraction 从 23 缩到 10。但 FP16 的致命问题是动态范围太小（5 位 exponent），训练时梯度很容易 overflow（溢出）或 underflow（下溢）。

⁴视频画面时间区间：08:15 – 08:45。

```

98 ...because you want to use some custom logic to set the values later
99 nn.init.trunc_normal_(x, mean=0, std=1, a=-2, b=2) # @inspect x

102 def tensors_memory():
103     Almost everything (parameters, gradients, activations, optimizer states) are stored as floating point numbers.

105 float32
106 [Wikipedia]
107 IEEE 754 single-precision 32-bit float
    sign | exponent (8 bit) | fraction (23 bit)
    0 0 1 1 1 1 1 0 0 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
    31 30 29 28 27 26 25 24 23 22 | 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

108 The float32 data type (also known as fp32 or single precision) is the default.
109 Traditionally, in scientific computing, float32 is the baseline; you could use double precision (float64) in some cases.
110 In deep learning, you can be a lot sloppier.

112 Let's examine memory usage of these tensors.
113 Memory is determined by the (i) number of values and (ii) data type of each value

```

Stanford

Figure 5: BF16 (brain float) : 保留 FP32 的 8 位 exponent，牺牲 fraction，专为深度学习设计⁵

BF16 (bfloat16, brain float) 是更好的选择。它的位结构是 1 sign + 8 exponent + 7 fraction——exponent 位数与 FP32 完全相同，因此动态范围与 FP32 一致，只是精度 (fraction) 更低。这意味着你可以把 FP32 数值直接截断 (truncate) 成 BF16 而不改变其量级，训练稳定性远好于 FP16。BF16 由 Google Brain 团队开发，因此得名“brain float”。

BF16 是训练的事实标准

BF16 把 exponent (动态范围) 看得比 fraction (精度) 更重要，因为深度学习对动态范围更敏感、对精度更宽容。在 H100 这类现代硬件上，BF16 的峰值算力远高于 FP32。本课与工业界的训练默认都用 BF16。

2.3 Tensor 的内存占用

Tensor 的内存占用非常简单，由两个量决定：

$$\text{memory} = \text{num_elements} \times \text{bytes_per_element}$$

例如创建一个 4×8 的 float32 tensor: 元素数 = 32, 每个元素 4 字节, 总内存 = 128 字节。可以用 `tensor.element_size()` (每个元素字节数) 和 `tensor.numel()` (元素总数) 来查询。

⁵视频画面时间区间: 08:45 – 09:15。

2.3.1 混合精度训练

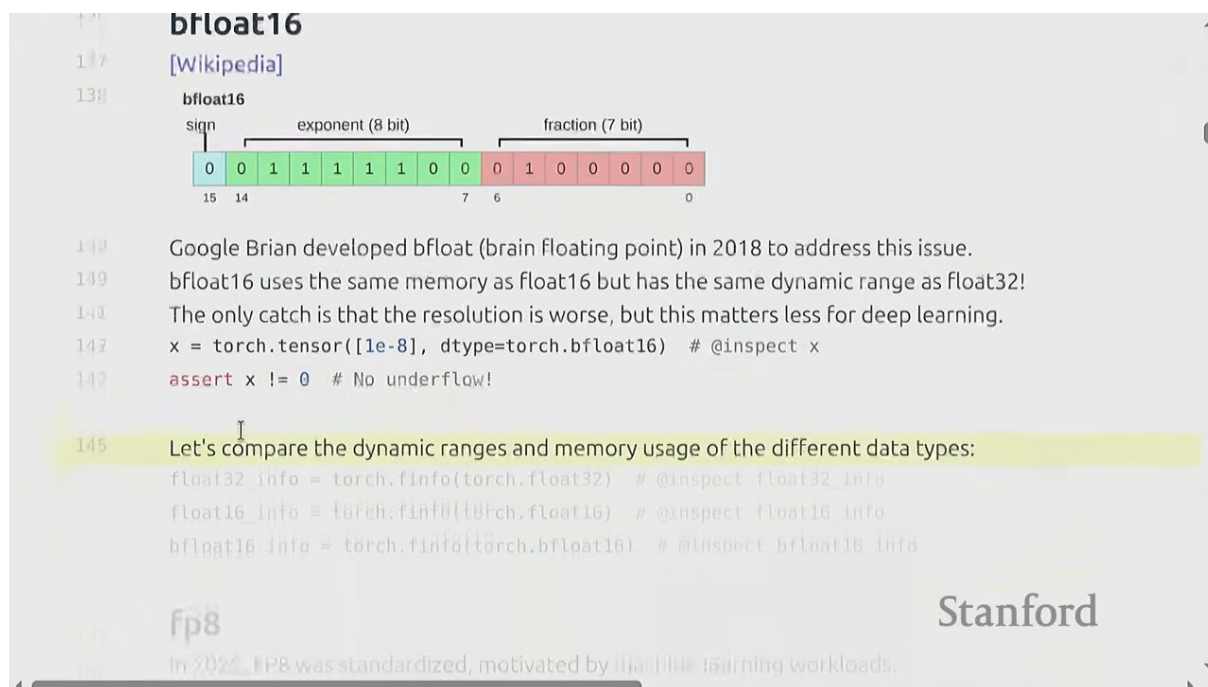


Figure 6: 混合精度训练：BF16 做前向/反向计算，FP32 保存 master weights 做参数更新⁶

Mixed precision training（混合精度训练）是工程上的折中。核心思路：用低精度（BF16/FP16）做前向和反向的计算以节省时间和内存，但保留一份 FP32 的 **master weights**（主权重副本）用于参数更新，保证数值稳定性。这是因为梯度累加和参数更新对精度敏感，若全程用低精度会导致训练不稳定。

什么时候用 FP32，什么时候用 BF16

- 前向/反向的大规模 matmul：用 BF16（快、省内存）
- 参数更新、梯度累加、optimizer state：用 FP32（稳定）
- 默认推荐：尽可能用 BF16 甚至 FP8 做前向，其余用 FP32

2.4 Views、Strides 与 Contiguous

理解 tensor 在内存中如何布局，对高效代码至关重要。PyTorch 的 tensor 并不总是连续存储——它通过 **strides**（步幅）来描述如何遍历数据。底层上，任何 tensor 的实际数据都存在一段一维的连续内存里，而 tensor 的“形状”只是一个逻辑视图：它告诉 PyTorch 如何用 strides 去索引这段一维存储。掌握这套机制，你就能预测哪些操作会拷贝数据（昂贵的）、哪些不会（便宜的），从而写出内存高效的代码。

⁶视频画面时间区间：10:45 – 11:15。


```
235 Get row 0:
236 y = x[0] # @inspect y
237 assert torch.equal(y, torch.tensor([1., 2, 3]))
238 assert same_storage(x, y)

240 Get column 1:
241 y = x[:, 1] # @inspect y
242 assert torch.equal(y, torch.tensor([2, 5]))
243 assert same_storage(x, y)

245 View 3x2 matrix as 2x3 matrix:
246 y = x.view(3, 2) # @inspect y
247 assert torch.equal(y, torch.tensor([[1, 2], [3, 4], [5, 6]]))
248 assert same_storage(x, y)

250 Transpose the matrix:
251 y = x.transpose(1, 0) # @inspect y
252 assert torch.equal(y, torch.tensor([[1, 4], [2, 5], [3, 6]]))
253 assert same_storage(y, x)
```

Stanford

Figure 7: Views 与 strides: reshape/view 不拷贝数据，仅改变逻辑形状与遍历步幅⁷

Strides 是一个元组，告诉你“在某个维度上前进一个位置需要在底层一维存储里跳过多少个元素”。例如一个 row-major（行主序）的 2×3 tensor，strides 是 (3, 1)：行方向前进 1 跳 3 个元素，列方向前进 1 跳 1 个元素。很多操作（transpose、reshape、slice）只改变 shape 和 strides，不拷贝数据——这叫 **view**。

```
252 assert torch.equal(y, torch.tensor([[1, 4], [2, 5], [3, 6]]))
253 assert same_storage(x, y)

255 Check that mutating x also mutates y.
256 x[0][0] = 100 # @inspect x, @inspect y
257 assert y[0][0] == 100

259 Note that some views are non-contiguous entries, which means that further views aren't possible.
260 x = torch.tensor([1., 2, 3], [4, 5, 6]) # @inspect x
261 y = x.transpose(1, 0) # @inspect y
262 assert not y.is_contiguous()
263 try:
264     y.view(2, 3)
265     assert False
266 except RuntimeError as e:
267     assert "view size is not compatible with input tensor's size and stride" in str(e)

269 One can enforce a tensor to be contiguous first:
270 y = x.transpose(1, 0).contiguous().view(2, 3) # @inspect y
271 assert not same_storage(x, y)
```

Stanford

Figure 8: Contiguous tensors: view() 需连续内存，reshape() 总能用⁸

⁷视频画面时间区间：20:45 – 21:15。

⁸视频画面时间区间：21:45 – 22:15。

view() vs reshape()

- `tensor.view(...)`: 要求 `tensor` 是 `contiguous` (连续的), 即元素在内存中按遍历顺序紧凑排列, 否则报错。
- `tensor.reshape(...)`: 总是可用。如果不能做 `view`, 它会自动拷贝数据。
- 经验法则: 想安全就 `reshape`; 想确保不拷贝、对性能敏感就用 `view` (并在必要时先调 `.contiguous()`)。

Percy 强调, 理解 `strides` 能让你读懂那些看起来很“魔法”的代码 (比如 `x[:, :-1]` 与 `x[:, 1:]` 共享底层存储), 也能帮你调试奇怪的形状错误。

2.5 Broadcasting

当两个 `tensor` 形状不同但要做逐元素运算时, PyTorch 用 **broadcasting** (广播) 规则自动扩展较小的 `tensor`。规则是: 从右向左对齐维度, 每个维度要么相等、要么其中一个为 1 (被扩展)、要么不存在 (被当作 1)。这让代码更简洁, 但也容易引入隐蔽的 bug——Percy 提醒要养成对每个 `tensor` 标注维度含义的习惯。

3 计算图与 Autograd

3.1 Autograd: 自动微分

```
171 Let's first see if we have any GPUs.
172 if not torch.cuda.is_available():
173     return
174
175 num_gpus = torch.cuda.device_count() # @inspect num_gpus
176 for i in range(num_gpus):
177     properties = torch.cuda.get_device_properties(i) # @inspect properties
178
179     memory_allocated = torch.cuda.memory_allocated() # @inspect memory allocated
180
181     Move the tensor to GPU memory (device 0).
182     y = x.to("cuda:0")
183     assert y.device == torch.device("cuda", 0)
184
185     Or create a tensor directly on the GPU:
186     z = torch.zeros(32, 32, device="cuda:0")
187
188     new_memory_allocated = torch.cuda.memory_allocated() # @inspect new memory allocated
189     memory_used = new_memory_allocated - memory_allocated # @inspect memory used
190     assert memory_used == 2 * (32 * 32 * 4) # 2 32x32 matrices of 4-byte floats
```

Figure 9: Autograd: `requires_grad=True` + `backward()` 自动计算梯度⁹

Percy 现场提醒一个常见误区: `backward()` 默认只释放计算图、保留叶节点的梯度。如果你想在非叶节点上检查中间梯度 (用于调试), 必须显式调用 `tensor.retain_grad()`, 否则反向传播一过该节点, 它的梯度就被丢弃以节省内存。理解这一点能帮你解释“为什么打印某个中间变量的 `.grad` 却得到 `None`”。

PyTorch 的 **autograd** (自动微分引擎) 是深度学习的核心。机制很简单:

1. 创建 `tensor` 时设 `requires_grad=True`, PyTorch 开始追踪对它的所有操作

⁹视频画面时间区间: 15:45 – 16:15。

2. 对这些操作，autograd 在后台构建一个 **computational graph**（计算图），记录每个操作及其反向求导方式
3. 调用 `loss.backward()` 后，autograd 沿计算图反向传播，用链式法则（chain rule）算出每个叶节点的梯度，存入 `tensor.grad`

例如 `y = x ** 2`；`y.backward()` 后，`x.grad` 就是 $\frac{dy}{dx} = 2x$ 。

3.2 计算图：leaf variables

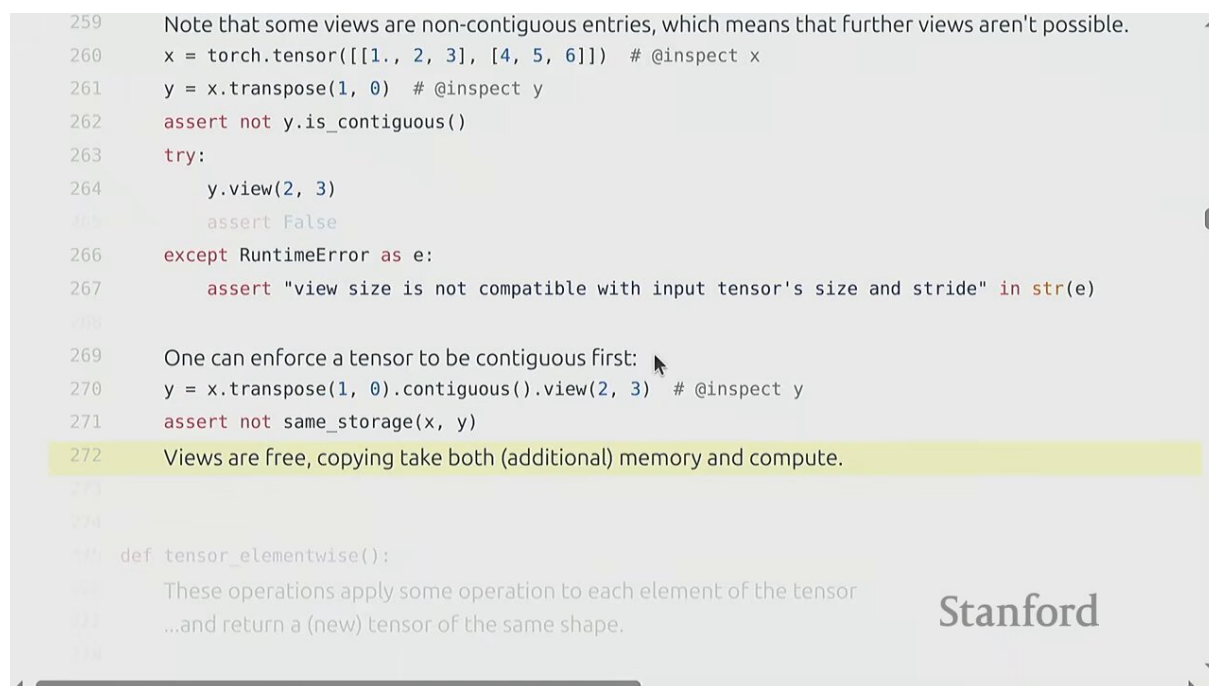


Figure 10: 计算图：leaf variables（用户创建的）与非叶子节点（操作产生的）¹⁰

计算图中的节点分两类。**Leaf variables**（叶变量）是用户直接创建的 `tensor`（如权重 W 、输入 x ），是梯度真正需要保留的目标。非叶子节点是操作产生的中间结果（如 $h = Wx$ ），它们的梯度在反向传播中被消费、默认不保留（节省内存）。如果想保留中间梯度，需调 `.retain_grad()`。

为什么这样设计

你真正要更新的是参数（叶变量），中间激活只是链式法则的“过路费”。默认丢弃中间梯度是内存优化。理解 leaf vs non-leaf 是排查“为什么我的 `.grad` 是 `None`”这类 bug 的关键。

¹⁰视频画面时间区间：22:45 – 23:15。

4 构建模型: nn.Module

4.1 nn.Linear: 最小的模块

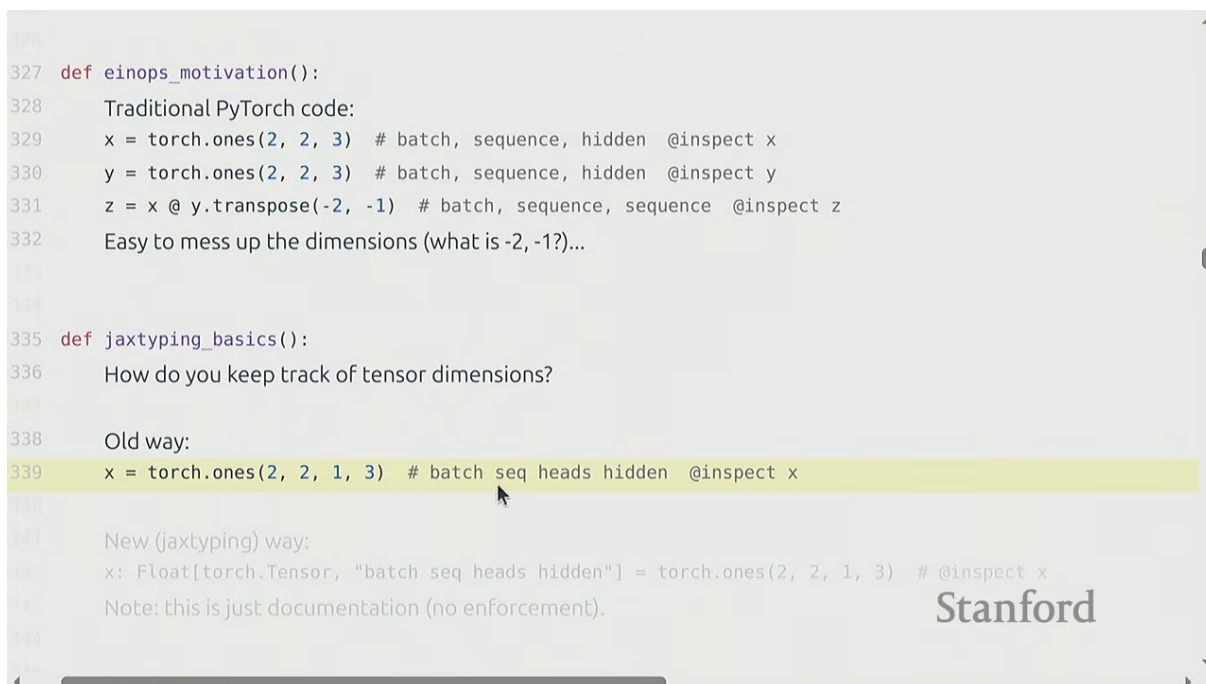


Figure 11: nn.Linear: weight 是 Parameter, forward 做 $F.linear(x, weight)$ ¹¹

nn.Linear 是最基础的模块。它持有一个权重矩阵 $W \in \mathbb{R}^{d_{out} \times d_{in}}$ (PyTorch 约定这样存), **forward** 就是 $y = xW^T$ (加可选 **bias**)。关键点: **W** 是 **nn.Parameter**——这是 **nn.Tensor** 的子类, 会被自动注册为“需要求梯度的模块参数”, 从而被 optimizer 发现和更新。

线性层的前向计算为:

$$h = xW^T, \quad x \in \mathbb{R}^{B \times d_{in}}, \quad W \in \mathbb{R}^{d_{out} \times d_{in}}, \quad h \in \mathbb{R}^{B \times d_{out}}$$

- x : 输入, 形状 (B, d_{in}) , B 为 batch (或 batch $\times$ sequence)
- W : 权重, 形状 (d_{out}, d_{in}) , 是可学习参数
- h : 输出, 形状 (B, d_{out})

¹¹视频画面时间区间: 26:45 – 27:15。

4.2 nn.Module: 组合的基石

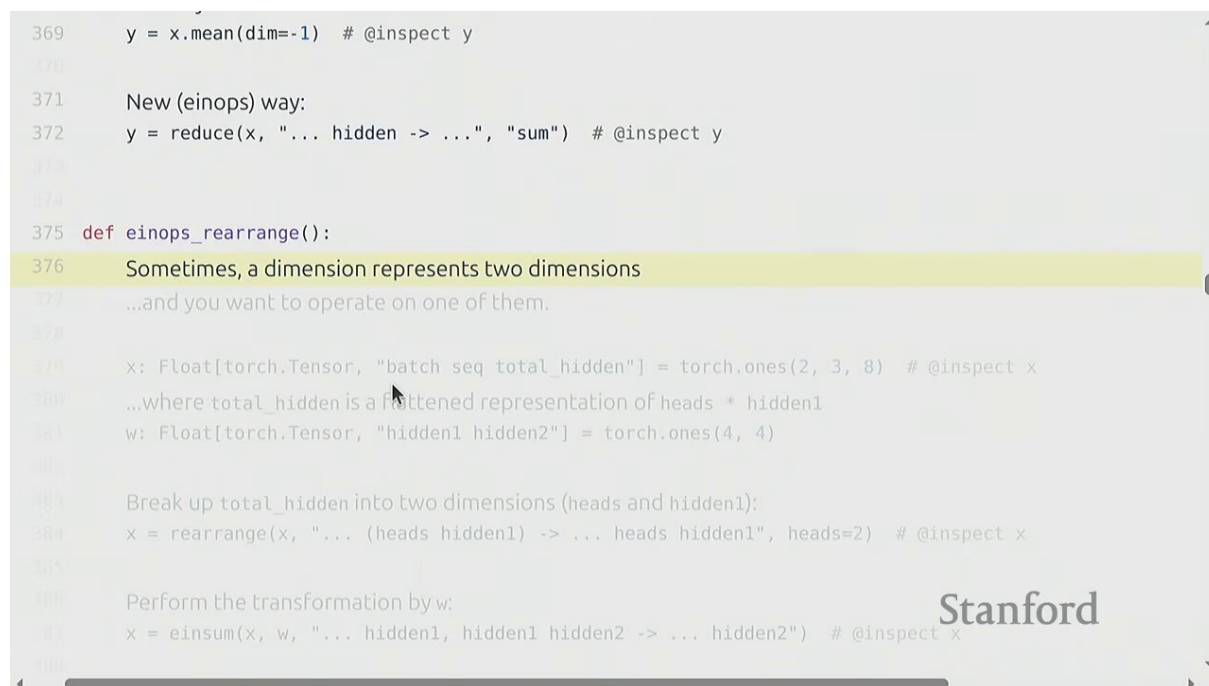


Figure 12: MLP 的 nn.Module 实现: `__init__` 定义子层, `forward` 描述计算¹²

`nn.Module` 是所有模型组件的基类。它的两个核心方法:

- `__init__(self, ...)`: 声明子模块和参数 (如 `self.fc1 = nn.Linear(...)`)
- `forward(self, x)`: 描述前向计算流程

`nn.Module` 自动追踪所有子模块和参数 (通过 `.parameters()` 迭代器暴露给 optimizer)。一个两层 MLP (Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ ReLU) 就是 `nn.Module` 的典型用法。这种“声明 + forward”的模式让模型定义既模块化又可读。

4.3 Einsum: 给维度起名字

PyTorch 代码里常见 `x @ y.transpose(-2, -1)` 这种用 `-2, -1` 索引维度的写法, 很容易出错且难读。Percy 推荐用 `einsum` (爱因斯坦求和约定, Einstein summation notation) 和 `jaxtyping` 库来给维度命名。例如:

```
torch.einsum("bs h, b s k h -> b s k", x, y)
```

直接写出 batch (b)、sequence (s)、hidden (h) 等维度名。einsum 的规则: 输出里出现的维度被遍历, 没出现的维度被求和。虽然初看冗长, 但比 `-2, -1` 更不易错, 也便于文档化。`jaxtyping` 进一步用类型注解强制维度一致性 (但 PyTorch 默认不强制, 需额外 checker)。

Percy 还提到一个实用技巧: 用 `...` (省略号) 表示“广播任意数量的前导维度”, 这样写出的 einsum 更紧凑、更通用。例如 `"... s h, ... s h -> ... s"` 可以同时处理带或不带 batch 维度的输入。养成给维度命名的习惯, 能让你的代码在面对 batch、sequence、head 等多维 tensor 时少出低级形状错误。

¹²视频画面时间区间: 30:45 – 31:15。

4.4 训练循环

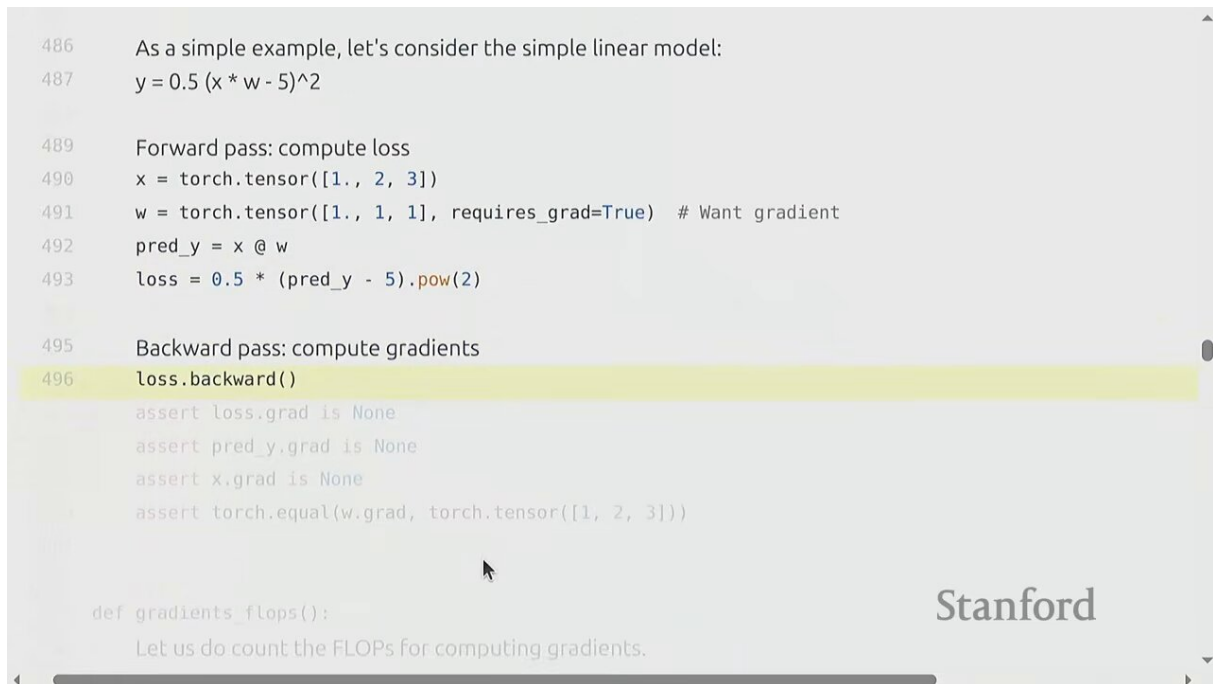


Figure 13: 标准训练循环: forward $\rightarrow$ loss $\rightarrow$ zero_grad $\rightarrow$ backward $\rightarrow$ step¹³

一个标准的 PyTorch 训练循环骨架如下:

```
1 for batch in dataloader:
2     inputs, labels = batch
3     outputs = model(inputs)           # []
4     loss = criterion(outputs, labels) # [] loss
5     optimizer.zero_grad()            # [] [] []
6     loss.backward()                   # [] [] []
7     optimizer.step()                  # [] [] []
```

容易忘的 `zero_grad()`

`optimizer.zero_grad()` 必须在每次 `backward()` 前调用。因为 PyTorch 默认把新梯度累加到 `.grad` 上 (这是 gradient accumulation 的基础)。如果你忘了清零, 梯度会一直累加, 训练立刻发散。

¹³ 视频画面时间区间: 50:45 – 51:15。

5 算力核算：从 matmul 到 6ND

5.1 矩阵乘法的 FLOPs

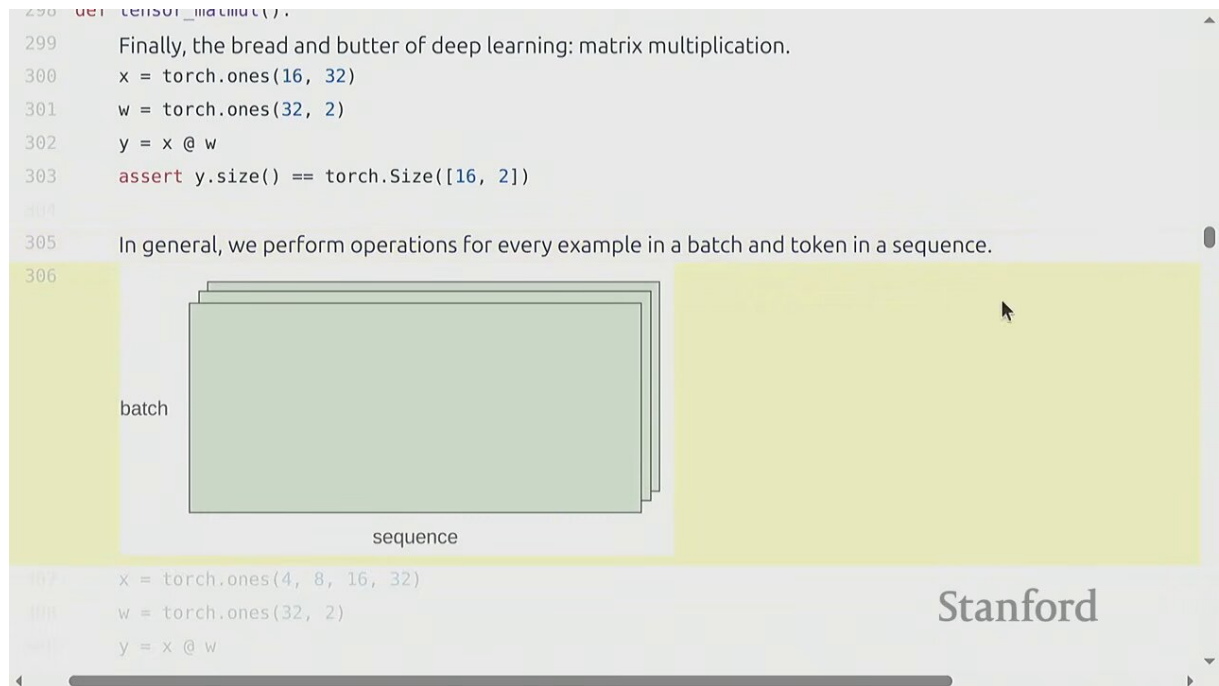


Figure 14: 矩阵乘法 FLOPs: $C = AB$, $(m \times k)(k \times n)$, 总 FLOPs = $2mnk$ ¹⁴

矩阵乘法 (matmul) 是深度学习的“面包黄油”。 $C = AB$, 其中 $A \in \mathbb{R}^{m \times k}$ 、 $B \in \mathbb{R}^{k \times n}$ 、 $C \in \mathbb{R}^{m \times n}$ 。 C 的每个元素是 A 的一行与 B 的一列的点积, 需要 k 次乘法和 $k - 1$ 次加法 $\approx 2k$ 次 FLOPs。 C 共 mn 个元素, 所以:

$$\text{FLOPs}_{\text{matmul}} = 2 \cdot m \cdot n \cdot k$$

- m, n, k : 两个矩阵的三个维度
- 系数 2: 每次乘加算作 2 次 FLOPs (1 乘 + 1 加)
- 这条 “2 × 所有维度乘积” 的规则是后续所有算力核算的基础

在语言模型里, matmul 通常带 batch 和 sequence 维度 ($B \times S \times \dots$), 但规则不变——每个 batch、每个 token 独立做 matmul, 总 FLOPs 就是再乘上 $B \cdot S$ 。

¹⁴视频画面时间区间: 24:15 – 24:45。

5.2 从 matmul 到 FLOPs/s 再到 MFU

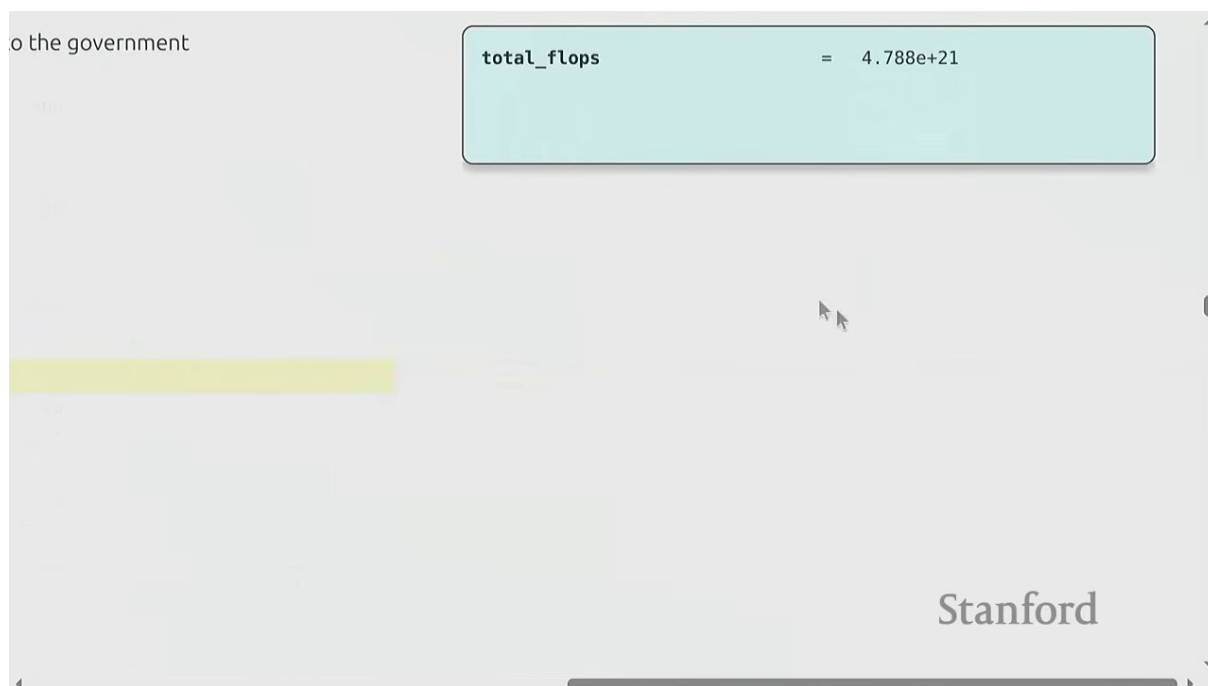


Figure 15: H100 峰值算力与每天秒数：989 TFLOPS (BF16)，每天 86400 秒¹⁵

知道了某个 matmul 的 FLOPs，再用 `time` 模块测它实际耗时，就能得到实际吞吐：

$$\text{actual FLOPs/s} = \frac{\text{FLOPs}_{\text{matmul}}}{\text{wall_time}}$$

把这个数和硬件的“宣传峰值”对比，就得到 **MFU**（model FLOPs utilization，模型算力利用率）：

$$\text{MFU} = \frac{\text{actual FLOPs/s}}{\text{promised FLOPs/s}}$$

- actual FLOPs/s：你实测的吞吐（模型有效计算量 / 实际耗时）
- promised FLOPs/s：硬件宣传的峰值（如 H100 的 989 TFLOPS BF16），来自厂家“光面宣传册”
- 注意：MFU 看的是模型的有效计算量，不是硬件实际执行的指令数——所以即使你用了 gradient checkpointing 多算了一遍前向，MFU 仍按“模型本该算的量”计，不会因为你“聪明”而惩罚你

MFU 的经验值

- MFU > 0.5：通常被认为是“好”的实现
- MFU $\approx$ 0.05（5%）：很差，说明有大把算力被浪费在通信、kernel 启动、内存搬运上
- 几乎不可能达到 0.9 – 1.0，因为这忽略了所有通信和开销，只算纯计算
- matmul 占主导时 MFU 通常更高
- 永远要 benchmark 你的代码，别假设能拿到宣传峰值

¹⁵视频画面时间区间：36:45 – 37:15。

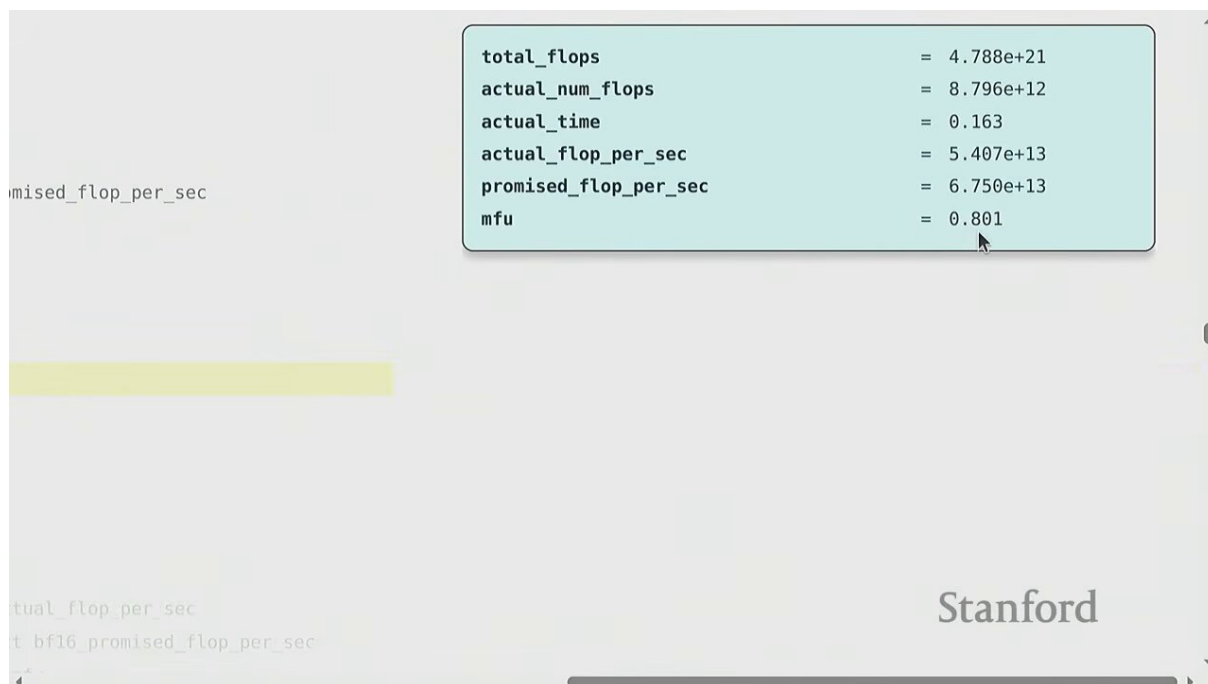


Figure 16: 训练时间估算完整公式: $6 \times 70B \times 15T / (1024 \times 989e12 \times 0.5 \times 86400) \approx 144$ 天¹⁶

把 MFU 串进来, 训练时间的完整公式为:

$$T_{\text{train}} = \frac{6 \cdot N \cdot D}{G \cdot P \cdot \text{MFU}}$$

- N : 模型参数量, D : 训练 token 数
- 分子 $6ND$: 总训练 FLOPs (前向 2 + 反向 4, 下节推导)
- G : GPU 数量, P : 单卡峰值 FLOPs/s
- 代入开场问题: $N = 70B, D = 15T, G = 1024, P = 989e12, \text{MFU} = 0.5$, 得 ≈ 144 天

Tensor Core 与 BF16 的关系

学生提问: PyTorch 默认会用 tensor core 吗? 答案是: tensor core 是 GPU 上专门做 matmul 的硬件单元, spec sheet 上那些高 FLOPs 数字都是基于 **tensor core** 的。用 BF16/F16 且配合 **torch.compile**, PyTorch 会生成调用 tensor core 的代码。FP32 路径的峰值则低得多 (如 H100 FP32 约 67 TFLOPS, BF16 约 989 TFLOPS)。

¹⁶视频画面时间区间: 44:45 – 45:15。

5.3 Linear Layer 的 FLOPs: 前向与反向

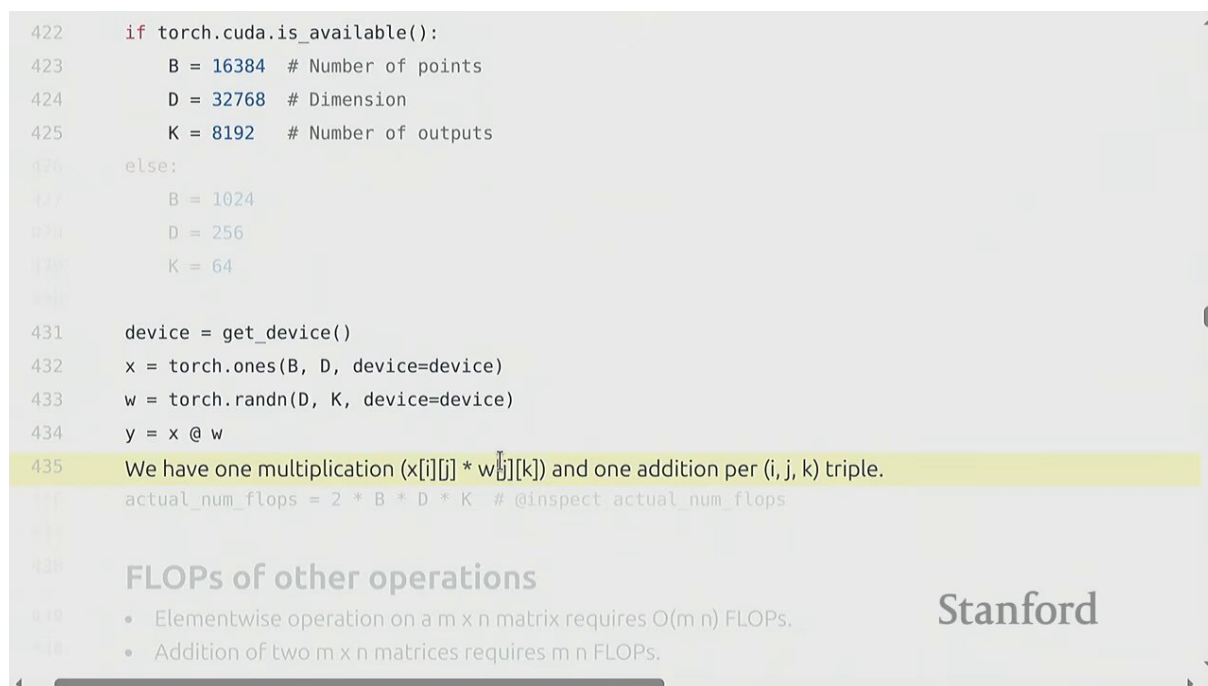


Figure 17: Linear layer FLOPs: 单 token 前向 $2d_{in}d_{out}$, 反向约 $4d_{in}d_{out}$ ¹⁷

考虑一个两层线性网络: $H_1 = XW_1$ ($X \in \mathbb{R}^{B \times D}$, $W_1 \in \mathbb{R}^{D \times D}$), $H_2 = H_1W_2$ ($W_2 \in \mathbb{R}^{D \times K}$), 再算 loss。

5.3.1 前向 FLOPs

前向就是两个 matmul:

$$\text{FLOPs}_{\text{fwd}} = 2 \cdot B \cdot D \cdot D + 2 \cdot B \cdot D \cdot K = 2 \cdot B \cdot (\text{参数量})$$

- 第一项 $2BDD$: XW_1 , 对应 W_1 的 $D \cdot D$ 个参数
- 第二项 $2BDK$: H_1W_2 , 对应 W_2 的 $D \cdot K$ 个参数
- 关键观察: 前向 $\text{FLOPs} = 2 \times \text{batch} \times \text{参数量}$

5.3.2 反向 FLOPs: 链式法则

反向传播要算 $\partial \text{loss} / \partial W_1, \partial \text{loss} / \partial W_2, \partial \text{loss} / \partial H_1, \dots$ 。以 W_2 为例, 链式法则给出:

$$\frac{\partial \text{loss}}{\partial W_2} = H_1^\top \frac{\partial \text{loss}}{\partial H_2}$$

这本身是一个 matmul, FLOPs 为 $2BDK$ 。但还要继续往前传 $\partial \text{loss} / \partial H_1$:

$$\frac{\partial \text{loss}}{\partial H_1} = \frac{\partial \text{loss}}{\partial H_2} W_2^\top$$

这又是一个 matmul, FLOPs 也是 $2BDK$ 。所以仅 W_2 这一层反向就需要 $2 \times 2BDK = 4BDK$ 。同理 W_1 层反向需要 $4BDD$ 。

¹⁷视频画面时间区间: 38:45 – 39:15。

前向 2，反向 4，总共 6

$$\text{FLOPs}_{\text{fwd}} = 2 \cdot B \cdot (\text{参数量}), \quad \text{FLOPs}_{\text{bwd}} = 4 \cdot B \cdot (\text{参数量})$$

$$\text{FLOPs}_{\text{total}} = 6 \cdot B \cdot (\text{参数量})$$

- 前向每个参数对应 ≈ 2 FLOPs（一次 matmul）
- 反向每个参数对应 ≈ 4 FLOPs（两个 matmul：算 $\partial/\partial W$ 和 $\partial/\partial H$ ）
- 总计 $6 \times \text{参数量} \times \text{数据点数}$ ——这正是开场公式 $6ND$ 中 6 的来源

5.3.3 推广到 Transformer

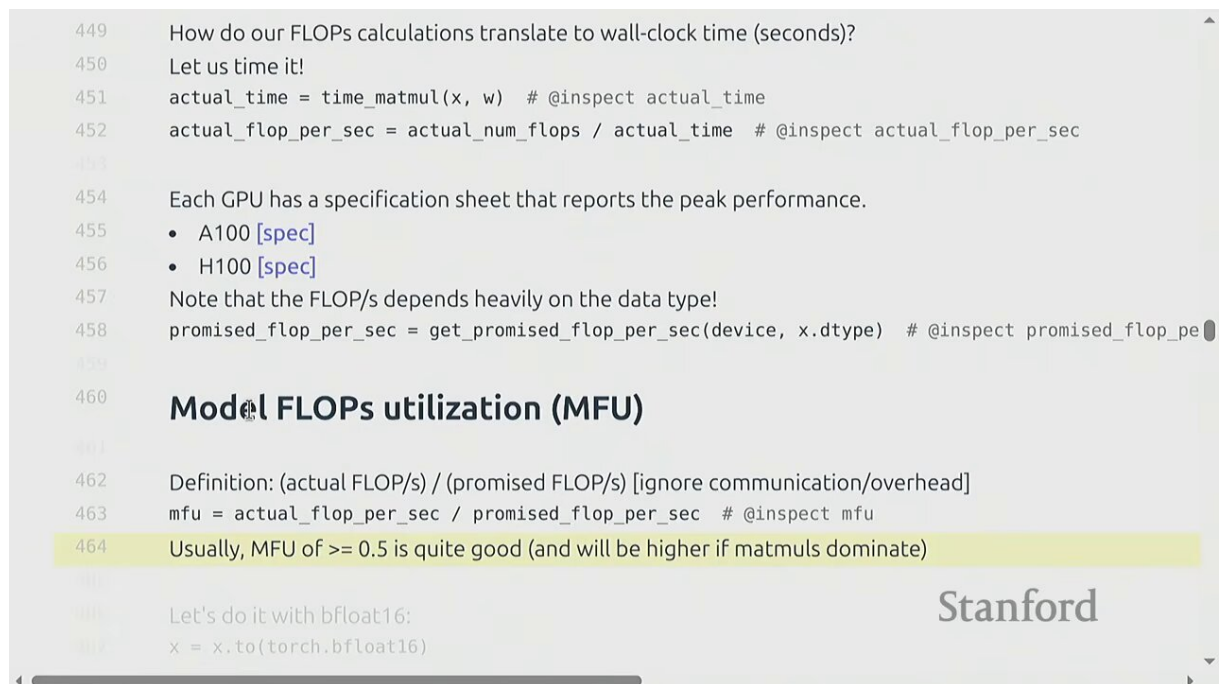


Figure 18: Transformer 训练总 FLOPs $\approx 6ND$ ：6 的来源是前向 2 + 反向 4¹⁸

这个 $6 \times$ 规则对绝大多数模型成立，因为大部分计算都是 matmul，且每个计算大致“触及”一组新参数。例外是参数共享（如一个参数被复用亿次，FLOPs 远超 $6N$ ），但正常模型不这样。因此 Transformer 的训练 FLOPs 也近似为：

$$\text{FLOPs}_{\text{transformer}} \approx 6 \cdot N_{\text{params}} \cdot N_{\text{tokens}}$$

¹⁸视频画面时间区间：46:45 – 47:15。

5.4 Attention 的 FLOPs

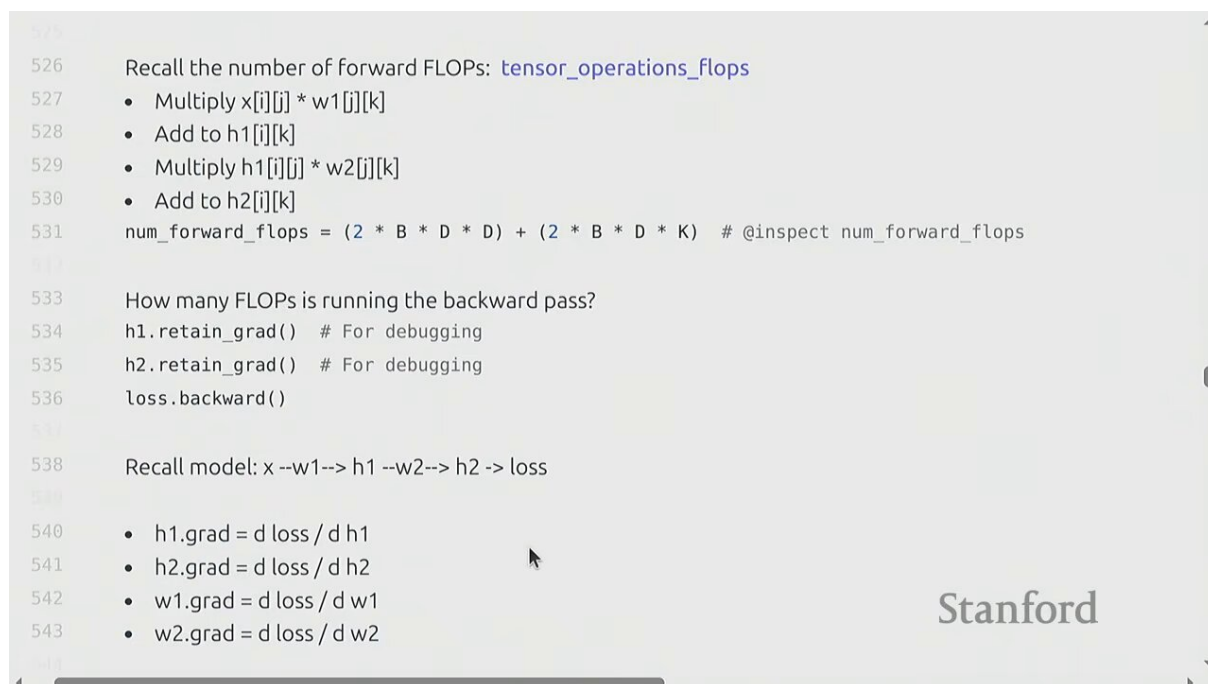


Figure 19: Attention FLOPs: $\text{softmax}(QK^\top/\sqrt{d})V$, QKV 投影主导计算量¹⁹

标准 attention 的计算为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

FLOPs 分解:

- $QK^\top$: $(B \times S \times d) \times (B \times d \times S) \rightarrow (B \times S \times S)$, FLOPs = $2BS^2d$
- $\text{softmax}(QK^\top)V$: $(B \times S \times S) \times (B \times S \times d) \rightarrow (B \times S \times d)$, FLOPs = $2BS^2d$
- softmax 本身的 FLOPs 相对可忽略
- 还有 Q, K, V 三个投影和输出投影, 每个是 $2BSd^2$

Attention 的两个 regimes

当序列长度 S 远小于隐藏维度 d 时, QKV 投影 ($O(Sd^2)$) 主导; 当 S 很大时, $QK^\top$ 和 $\text{softmax}V$ ($O(S^2d)$) 主导——这正是长序列的瓶颈, 催生了 FlashAttention、sliding window 等优化。在第一讲提到的 175B 模型上, MLP 的 FLOPs 占主导 (约 2/3), attention 相对小。

6 内存核算: 四类占用

讲完算力, 回到内存。一个训练中的模型, 内存被四类对象占用。

6.1 参数与梯度

Parameters (参数): 模型权重本身。对一个 L 层、隐藏维度 d 的简单模型, 参数量约为 $L \cdot d^2 + d$ (最后一层 head 是 d)。

Gradients (梯度): 与参数一一对应, 形状和数量完全相同。所以梯度占用 = 参数占用。

¹⁹视频画面时间区间: 54:45 – 55:15。

6.2 激活内存

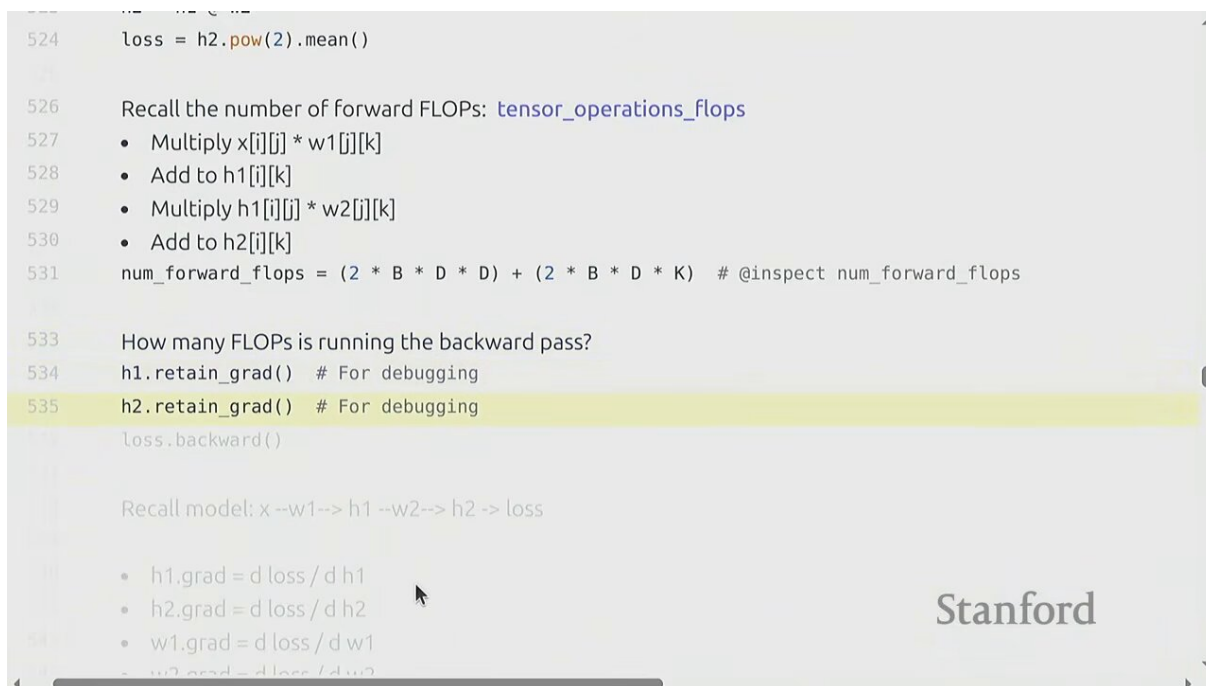


Figure 20: 激活内存：随层数线性增长，每层激活需缓存供反向传播²⁰

Activations（激活）：反向传播时，第 i 层的梯度依赖于第 i 层的输入激活。所以前向时必须把每层激活缓存下来，留到反向时用。激活占用为：

$$\text{memory}_{\text{act}} = B \cdot S \cdot d \cdot L \cdot (\text{bytes_per_element})$$

- B : batch size, S : 序列长度, d : 隐藏维度, L : 层数
- 每个数据点、每个 token、每层都要存一个 d 维向量
- 激活内存随层数 L 线性增长——这是深层模型的主要内存瓶颈

为什么要存激活

反向传播算第 i 层权重的梯度需要第 i 层的输入。朴素做法是全存下来。但有更聪明的办法——下节的 gradient checkpointing。

²⁰视频画面时间区间：52:45 – 53:15。

6.3 优化器状态内存

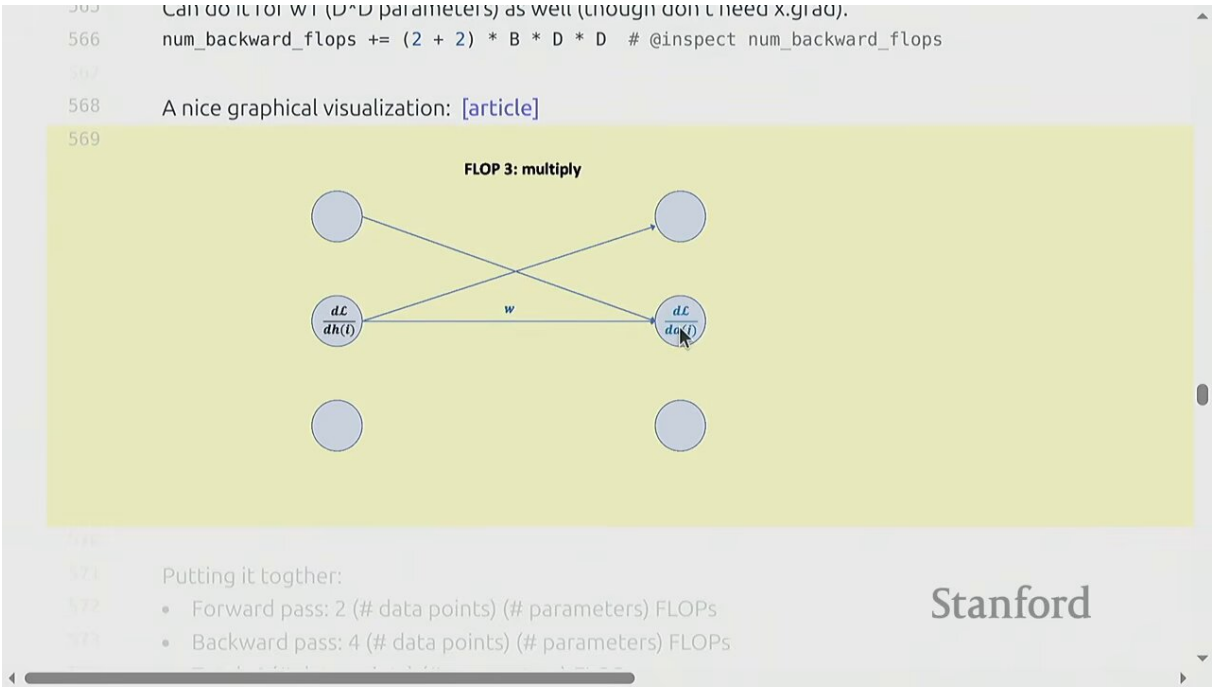


Figure 21: 优化器状态内存：AdamW 需存 m 和 v （各 FP32），每参数约 12 – 16 字节²¹

Optimizer state（优化器状态）：这是最容易被忽略、却往往占用最大的一块。不同优化器状态量不同：

优化器	状态量（每参数）	说明
SGD（无 momentum）	0	只用梯度，无额外状态
SGD with momentum	1 份（velocity v ）	累加梯度的滑动平均
Adagrad	1 份（ $G = \sum g^2$ ）	累加历史梯度平方和
RMSprop	1 份（ $E[g^2]$ ）	Adagrad 的指数衰减版
Adam / AdamW	2 份（ m 和 v ）	一阶矩 + 二阶矩的指数滑动平均

AdamW 的每参数内存（FP32）

$$\text{bytes/param} = \underbrace{4}_{\text{param}} + \underbrace{4}_{\text{grad}} + \underbrace{4}_m + \underbrace{4}_v = 16 \text{ 字节}$$

若用混合精度（参数存 BF16 但 optimizer state 存 FP32）：参数 2 + 梯度 2 + m 4 + v 4 = 12 字节。这就是开场“16 字节/参数”和“8 张 H100 能训 40B”的由来。

6.4 总内存公式

把四类加起来：

$$\text{memory}_{\text{total}} = (\text{params} + \text{grads} + \text{optim_state}) \times \text{bytes} + \text{activations}$$

- 前三项正比于参数量 N ，与 batch/sequence 无关
- activations 正比于 $B \cdot S \cdot d \cdot L$ ，随 batch 和序列长度增长
- Assignment 1 要求对 Transformer 做这个分解——形式相同，只是矩阵更多

²¹ 视频画面时间区间：56:45 – 57:15。

Percy 用一个具体数字收尾这个例子：对一个 $d = 2048$ 、 $L = 8$ 层、 $B = 64$ 的模型，参数部分约占几百兆字节，而激活部分也达到同样量级——这说明在中等规模下激活就已经和参数相当，深层大模型时激活甚至会超过参数。这也是为什么 gradient checkpointing、混合精度等技术如此重要：它们直接削减的是这四类占用中最大的一块。

7 Optimizer 的实现

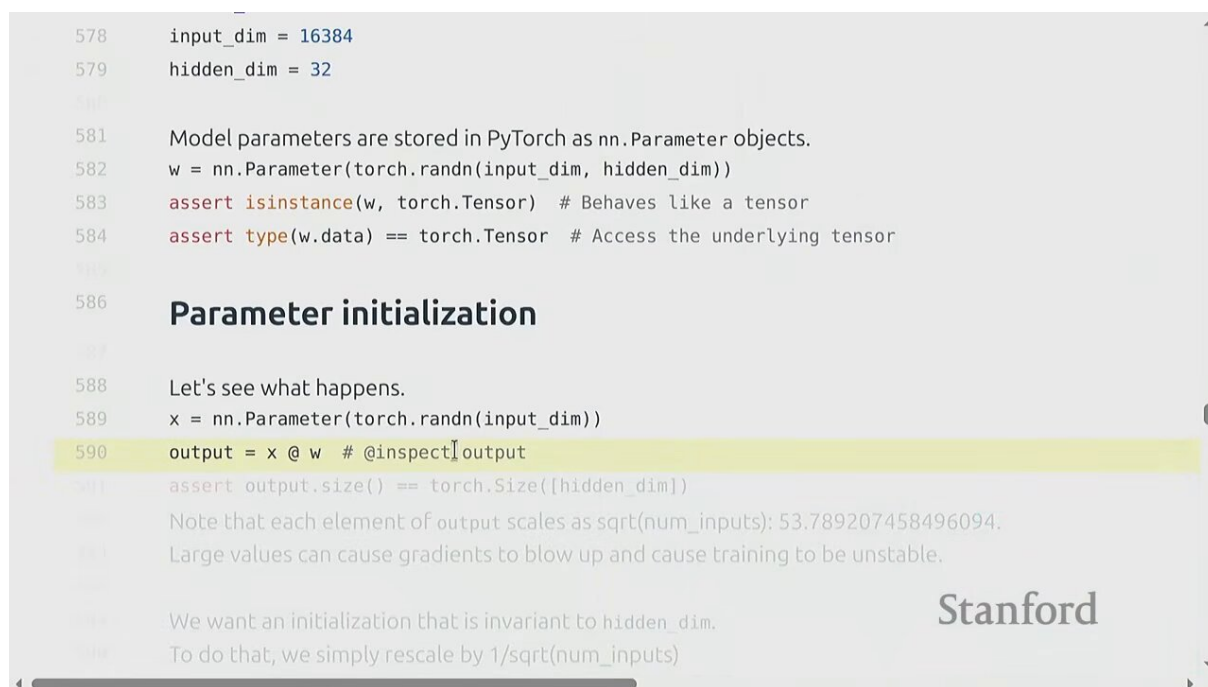


Figure 22: 优化器实现：继承 Optimizer，在 step() 里读梯度、更新状态、改参数²²

在 PyTorch 里实现自定义 optimizer，需继承 `torch.optim.Optimizer` 并重写 `step()`。Percy 现场演示了 Adagrad 的实现（因为 Assignment 1 要实现 Adam，所以课上讲 Adagrad 避免剧透）。

Adagrad 的更新规则：维护一个累积量 $G = \sum_t g_t^2$ （逐元素），更新时：

$$G_t = G_{t-1} + g_t \odot g_t, \quad \theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

- G_t ：历史梯度平方和（optimizer state，每个参数一份）
- g_t ：当前梯度（由 `backward()` 算好，存在 `param.grad`）
- η ：学习率， ϵ ：防除零小常数
- $\odot$ ：逐元素乘除

优化器的演进谱系

SGD $\rightarrow$ momentum（加滑动平均） $\rightarrow$ Adagrad（按参数自适应步长，但 G 单调增导致步长趋零） $\rightarrow$ RMSprop（用指数衰减代替单调累加） $\rightarrow$ Adam（2014，结合 momentum + RMSprop，同时维护 m 和 v ） $\rightarrow$ AdamW（解耦 weight decay）。Adam 是本课和当下的默认选择。

`step()` 的关键模式：通过 `self.param_groups` 拿到参数分组，对每个参数读 `param.grad`，更新存在 `self.state[param]` 里的优化器状态，最后写出新的 `param.data`。状态跨多次 `step()` 调用持久化。Percy 强调一个细节：optimizer 假设梯度已经被 `backward()` 算好并填进 `param.grad`，它本身不负责求梯度——分工明确：autograd 算梯度，optimizer 用梯度改参数。理解这个职责边界，有助于你在调试时定位问题到底出在前向、反向还是更新阶段。

²²视频画面时间区间：60:45 – 61:15。

8 Gradient Checkpointing 与实用技巧

8.1 Gradient Checkpointing: 用计算换内存

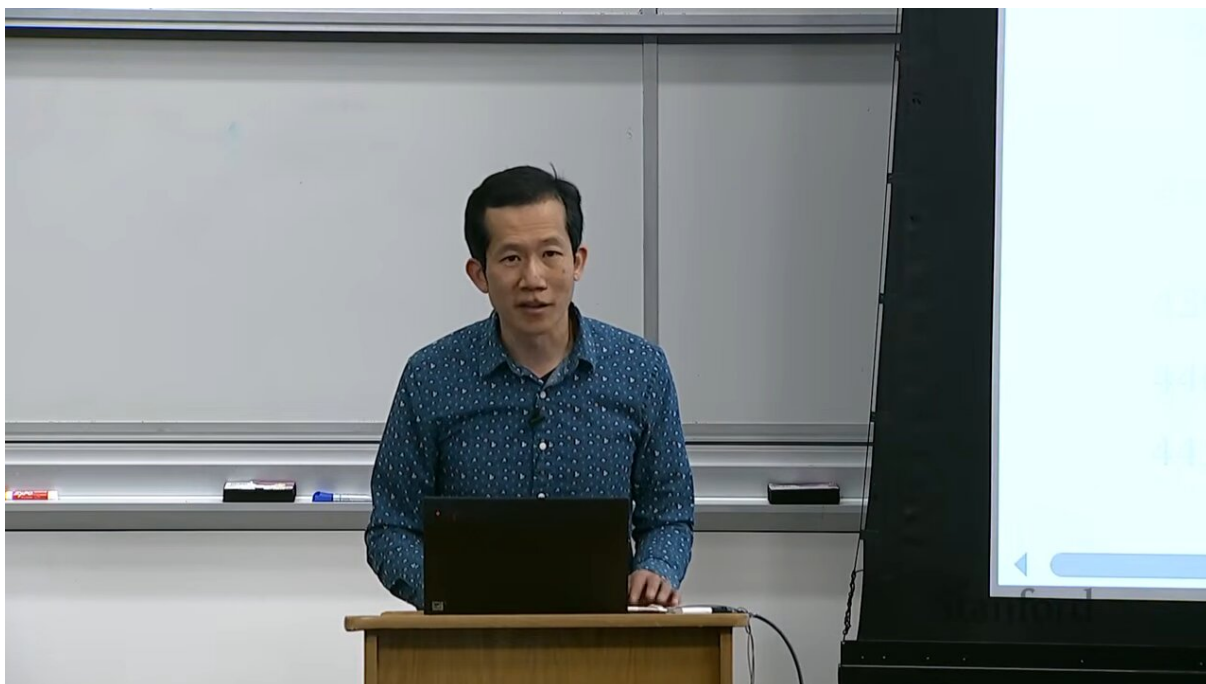


Figure 23: Gradient checkpointing: 只存 $\sqrt{n}$ 个检查点, 反向时重算, 内存从 $O(n)$ 降到 $O(\sqrt{n})$ ²³

Gradient checkpointing (梯度检查点, 又称 activation checkpointing 或 recomputation) 是解决激活内存爆炸的关键技术。朴素训练要存所有层的激活 ($O(n)$, n 为层数)。checkpointing 的思路: 前向时只存少数几个检查点层的输入, 其余激活丢弃; 反向传播到某层时, 从最近的检查点重新做一次前向算出需要的激活。

以计算换内存

baseline : memory = $O(n)$, compute = $1 \times \text{fwd}$

checkpointing : memory = $O(\sqrt{n})$, compute $\approx 1.5 \times \text{fwd}$

- 内存从层数线性 $O(n)$ 降到约 $O(\sqrt{n})$ (最优检查点策略下)
- 代价是多算约一次前向 (额外 $\sim 33\%$ 计算), 即“用计算换内存”
- 在内存受限 (如长序列、大 batch) 时极其有用, 是训练超大模型的标配

8.2 Checkpointing (模型保存)

两种 checkpointing 别混淆

- Gradient checkpointing (上节): 训练时省内存的技术
- Model checkpointing (本节): 把训练状态存盘以防崩溃

训练语言模型耗时极长, 一定会崩溃。为不丢失进度, 需周期性地把状态存盘。要存的不仅仅是模型权重, 还包括:

- 模型权重 (`model.state_dict()`)

²³ 视频画面时间区间: 40:45 – 41:15。

- 优化器状态 (`optimizer.state_dict()`) —— 否则恢复后 momentum/Adam 状态丢失，训练会抖动
- 当前迭代步数、学习率调度器状态等

恢复时把三者都 load 回来，从断点继续。这是工业级训练的基本功。

8.3 混合精度的实践建议

精度选择的工程经验

- 默认用 FP32 跑通流程
- 尽可能切到 BF16 (甚至 FP8) 做前向，FP32 留给参数更新
- PyTorch 提供 `torch.cuda.amp` / `autocast` 自动管理精度切换，免去手动指定每层的 dtype
- 前沿方向：FP8 全程训练（数值很挑，需要 trick），甚至 INT4；推理时可以做更激进的 quantization（量化），因为训练比推理对精度敏感得多

系统与模型协同设计

低精度训练的不稳定，部分要靠模型架构来缓解（如更稳的初始化、归一化）。反过来，Nvidia 芯片对低精度（INT4/FP8）有专门加速，这反过来驱动模型设计去适配硬件。这正是第一讲说的“协同设计数据、系统、模型”——Transformer 本身就是 GPU 友好性的产物，未来的架构演进会继续被硬件塑造。

9 总结

9.1 核心要点

本讲四大要点

1. 资源核算是心态：要高效，必须精确知道每份 FLOPs 和每字节内存去了哪里——它们直接换算成美元。Napkin math 让你在动代码前就能判断可行性。
2. 训练 FLOPs $\approx 6ND$ ：前向 2 + 反向 4。这个 6 来自链式法则——反向传播每个 matmul 权重要算两个梯度（对 W 和对 H ），所以反向是前向的两倍。
3. 内存四件套：参数 + 梯度 + 优化器状态 + 激活。前三项正比于参数量（AdamW 下约 16 字节/参数），激活正比于 $B \cdot S \cdot d \cdot L$ 。激活随层数线性增长是深层模型的内存瓶颈。
4. BF16 + 混合精度是训练默认：动态范围（exponent）比精度（fraction）更重要。Gradient checkpointing 用 $\sim 33\%$ 额外计算把激活内存从 $O(n)$ 降到 $O(\sqrt{n})$ 。

9.2 与 Assignment 1 的衔接

本讲用最简单的线性模型讲清了 resource accounting 的形式。Assignment 1 会让你对真正的 Transformer 做同样的分解——矩阵更多、有 attention，但骨架完全一样：参数、激活、梯度、优化器状态四类内存； $6ND$ 的 FLOPs。Percy 强调，亲手在 Transformer 上算一遍，这些概念才会真正扎实。

9.3 下节预告

下一讲 Tatsu 会从概念层面讲解 Transformer 架构。届时你将带着本讲建立的“每个 matmul 多少 FLOPs、每层多少激活”的核算直觉，去理解为什么 Transformer 这么设计、哪些变体在什么 scale 下有意义。

9.4 配套阅读

材料	对应知识点
PyTorch 官方 autograd 教程	autograd、计算图
Mixed Precision Training (Micikevicius 2017)	混合精度训练
Adam (Kingma 2014)	Adam 优化器
Decoupled Weight Decay / AdamW (Loshchilov 2017)	AdamW
Training Deep Nets with Sublinear Memory (Chen 2016)	Gradient checkpointing
H100 / A100 spec sheet	峰值 FLOPs、tensor core